

First, the basic idea

When students hear “High Level Diagram” and “Low Level Diagram”, they can map them to:

- **High Level Design (HLD)** → Big picture: modules, major components, data flow, external systems, main technologies.^{[1][2]}
- **Low Level Design (LLD)** → Detailed picture: classes, methods, attributes, database tables, exact interactions, algorithms.^{[3][4][1]}

Think of it like planning a building:

- HLD = layout of floors, rooms, lifts, parking.
- LLD = detailed drawings for electrical wiring, plumbing, beam sizes.

For final-year Python projects, both are needed: first HLD to show what you are building, then LLD to show how exactly you will code it.^{[5][1]}

Step 1: Pick a simple Python project example

To make it concrete, imagine a very simple project:

Project idea: “Library Management System” in Python (console or simple web app).

Main features:

- Students can search books.
- Librarian can add/remove books.

- System can issue/return books and track due dates.

We will build:

1. A **High Level Diagram** (HLD) for this.
2. A **Low Level Diagram** (LLD) for this.

Step 2: High Level Diagram – Big picture view

2.1 What does HLD show?

HLD usually shows:^{[2][1]}

- Main **modules/components**.
- How **data flows** between them.
- Any **external systems** (e.g., database).
- Overall **technology choices** (Python app, DB, UI).

You do **not** show individual classes or functions here.

2.2 HLD for Library Management System

For this simple project, you can define components like:

- **User Interface**
 - CLI (command line) or Web UI (Flask/Django).
- **Application Layer (Business Logic)**
 - Handles operations like search, issue, return.
- **Data Access Layer**

- Reads/writes data to the database or files.
- **Database / Storage**
 - Tables or JSON/CSV files storing books, users, issued records.

In a diagram (you can draw this on whiteboard or in any diagram tool):

- A box “UI”
- A box “Application Logic”
- A box “Data Access Layer”
- A cylinder “Database”

Arrows:

- UI → Application Logic (user sends commands).
- Application Logic → Data Access Layer (asks for data).
- Data Access Layer → Database (queries/updates).

This is your **High Level Diagram**. It answers:

“Which big parts exist in my system, and how they talk to each other?”^{[1][2]}

Step 3: Low Level Diagram – Code-level view

3.1 What does LLD show?

LLD uses UML diagrams like **class diagrams**, **sequence diagrams**, **activity diagrams** to describe:^{[4][3][1]}

- Classes and their **attributes** and **methods**.
- Relationships between classes (association, inheritance, composition).

- Step-by-step interaction flow (which object calls which method).

For a Python project, the most useful is usually the **class diagram**.^{[4][1]}

3.2 Identify classes from the HLD

From our Library example, convert real entities into classes:

- Book
- User (can later split into Student and Librarian if needed)
- Library (manages books and issuing)
- Transaction or IssueRecord (to track who borrowed what, and when)

Now, think of **responsibilities**: which class should do what?^{[3][1]}

- Book → store details: id, title, author, status (available/issued).
- User → store user_id, name, type (student/librarian).
- IssueRecord → store book_id, user_id, issue_date, due_date, return_date.
- Library → methods: add_book, remove_book, search_book, issue_book, return_book.

3.3 LLD: Simple Class Diagram (text form)

A typical UML class diagram (you would draw boxes, but here is a textual view inspired by UML class diagram structure):^[4]

- **Book**
 - Attributes: book_id, title, author, is_available
 - Methods: maybe mark_issued(), mark_returned()
- **User**

- Attributes: `user_id`, `name`, `user_type`
- Methods: none or simple helpers
- **IssueRecord**
 - Attributes: `record_id`, `book_id`, `user_id`, `issue_date`, `due_date`, `return_date`
 - Methods: maybe `mark_returned(date)`
- **Library**
 - Attributes: collections like `books`, `users`, `issue_records`
 - Methods:
 - `add_book(book)`
 - `remove_book(book_id)`
 - `search_books(query)`
 - `issue_book(book_id, user_id)`
 - `return_book(book_id, user_id)`

Relationships:

- Library “has many” Books.
- Library “has many” Users.
- Library “has many” IssueRecords.
- IssueRecord “refers to” one Book and one User.

If you draw this:

- Each class is a box with 3 parts: name, attributes, methods.^[4]

- Lines between classes show relations (plain line = association, line with multiplicity like 1..* to show “has many”).^{[1][4]}

3.4 How LLD connects to actual Python code

From the class diagram, you can now write Python classes (simplified idea, not full code):

- One Python file `models.py` containing `Book`, `User`, `IssueRecord`.
- One file `library.py` containing `Library` class.
- One file `main.py` which handles UI and uses `Library`.

This mapping “diagram → classes → Python files” is the essence of LLD.^{[3][1]}

Step 4: Compare High Level vs Low Level for this example

Aspect	High Level Diagram (HLD)	Low Level Diagram (LLD)
Focus	Architecture and major components ^{[1][2]}	Classes, methods, attributes, relationships ^{[1][3][4]}
Detail level	Coarse: UI, Logic, Data Access, DB	Fine: <code>Book</code> , <code>User</code> , <code>Library</code> , methods like <code>issue_book</code>
Who uses it	Stakeholders, mentors, teammates to understand overall system ^{[1][2]}	Developers who will actually implement code ^{[1][3]}

Typical diagrams	Component diagram, deployment diagram, simple block diagram [1][2]	UML class diagram, sequence diagram, activity diagram [1][4]
When in project	Early: after requirements, before coding [1][5]	After HLD is stable, just before/while coding [1][5]

Step 5: A simple step-by-step exercise for your students

You can ask your final-year batch to practice this in 5 steps:

1. Choose a simple Python project idea

Example: Library System, To-Do App, Bus Reservation, Expense Tracker.

2. Write 5–7 main features

This clarifies requirements before design.

3. Draw the High Level Diagram

- Identify 3–5 main components.
- Draw boxes and arrows showing how data flows.

4. Extract main classes from the HLD

- Pick nouns: Book, User, Ticket, Account, etc.
- Decide responsibilities and basic methods per class.

5. Draw the Low Level Class Diagram

- For each class: list attributes and methods.
- Draw relationships between classes.

- Then map each class to a Python file and start implementing.

This way, they see a clear path: **Requirements** → **HLD** → **LLD** → **Python code**.^{[6][1][3]}

To make this really useful, which project idea are your students actually considering for their Python project (for example, “online food ordering”, “student attendance system”, “chat app”)? If you share one concrete idea, the next step can be to help them build HLD and LLD for that exact project.

*
**

1. <https://www.geeksforgeeks.org/system-design/what-is-low-level-design-or-lld-learn-system-design/>
2. <https://www.youtube.com/watch?v=SKNlKMaxsT0>
3. <https://www.scribd.com/document/901987402/Low-Level-Design-Programming-in-Python-From-Basic>
4. <https://www.youtube.com/watch?v=OHfP9SthhTE>
5. <https://www.youtube.com/watch?v=-R668XYWNi0>
6. <https://docs.python-guide.org/writing/structure/>
7. <https://github.com/abhaypaswan/lld-python>
8. https://www.youtube.com/watch?v=gFGM_QvDl2M

9. https://digitalcollections.ohsu.edu/nanna/record/122/files/122_etd.pdf?withWatermark=0&withMetadata=0®isterDownload=1&version=1
10. <https://www.tnstate.edu/faculty/fyao/COMP3050/Py-tutorial.pdf>